

TREINAMENTO E

REGULARIZAÇÃO DE REDES NEURAIS



Backpropagation • Gradiente Descendente • Dropout • Ridge & Lasso • Early Stopping

FIAP — MBA em Data Science

Feature Engineering — Aula Prática

AGENDA DA AULA

01

Contexto de Negócio

Previsão de churn — por que treinar bem uma rede importa



02

Backpropagation

Como a rede aprende: forward pass, loss e chain rule



03

Gradiente Descendente

SGD, Mini-batch, Adam — otimizadores na prática



04

Overfitting & Regularização

Dropout, Ridge (L2) e Lasso (L1) em redes neurais



05

Early Stopping

Parando no momento certo para generalizar



06

Hands-On Integrado

Pipeline completo com TensorFlow/Keras no Jupyter





CONTEXTO DE NEGÓCIO

Por que o treinamento correto da rede neural impacta diretamente o resultado

O PROBLEMA: CHURN EM TELECOM

Cenário

Uma operadora de telecom perde **R\$ 4.200/cliente** que cancela o plano.

Dataset: 7.000 clientes, 21 features (tenure, contratos, serviços, billing).

Objetivo: construir uma rede neural que preveja churn com alta generalização — usando técnicas de treinamento e regularização corretas.

Por que isso importa?

26%

taxa de churn atual

R\$ 7.6M

perda anual estimada

12%

meta: reduzir churn p/ 12%



Um modelo com overfitting prevê bem nos dados de treino, mas falha em produção — custando milhões.



BACKPROPAGATION

O algoritmo que permite a rede neural aprender com seus erros

FORWARD PASS — A PREVISÃO

Dados de entrada percorrem a rede camada a camada até gerar uma previsão.



Cada neurônio calcula:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \quad \rightarrow \quad a = \sigma(z) \quad (\text{ativação: ReLU, Sigmoid, etc.})$$



No nosso caso de churn: as 21 features (tenure, contrato, billing...) entram na rede e o output é P(churn).

FUNÇÃO DE PERDA (LOSS)

Mede o quão errada está a previsão da rede em relação ao valor real.

Binary Cross-Entropy

Usada em classificação binária (churn: sim/não)

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

y = valor real (0 ou 1)

$\hat{y}$ = probabilidade prevista pela rede

Mean Squared Error (MSE)

Usada em regressão (prever valores contínuos)

$$L = (1/n) \sum (y_i - \hat{y}_i)^2$$

Penaliza erros grandes quadraticamente

Sensível a outliers



A loss diz QUANTO a rede errou. O backpropagation diz COMO corrigir. O gradiente descendente diz QUANTO corrigir.

BACKPROPAGATION — CHAIN RULE

A partir da loss, calculamos o gradiente (derivada parcial) camada a camada, de trás para frente.

1

Calcular Loss

Comparar $\hat{y}$ (previsão) com y (real)

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

2

Derivar de trás p/ frente

Regra da cadeia: $\partial L / \partial w = \partial L / \partial a \cdot \partial a / \partial z \cdot \partial z / \partial w$

$\partial L / \partial w$ (quanto cada peso contribuiu para o erro)

3

Atualizar pesos

$w_{\text{novo}} = w_{\text{antigo}} - \eta \cdot \partial L / \partial w$

η (learning rate) controla o tamanho do passo

Backpropagation = Chain Rule + Cálculo de Gradientes. É o "GPS" que diz a direção para reduzir o erro.



GRADIENTE DESCENDENTE

Otimizadores: como ajustar os pesos da rede de forma eficiente

VARIANTES DO GRADIENTE DESCENDENTE

Batch GD

Usa TODOS os dados para calcular o gradiente em cada step.

$$\theta = \theta - \eta \cdot \nabla L(\theta; X_{\text{todo}})$$



Convergência estável



Lento para datasets grandes

Stochastic GD (SGD)

Usa 1 amostra por vez. Ruidoso, mas rápido.

$$\theta = \theta - \eta \cdot \nabla L(\theta; x_i)$$



Rápido, escapa mínimos locais



Convergência instável

Mini-Batch GD

Usa lotes (32, 64, 128 amostras). Melhor dos dois mundos.

$$\theta = \theta - \eta \cdot \nabla L(\theta; X_{\text{batch}})$$



Eficiente + estável



Precisa tunar batch size

Na prática, usamos Mini-Batch GD com otimizadores como Adam, que adapta o learning rate automaticamente.

LEARNING RATE — O HIPERPARÂMETRO MAIS CRÍTICO

η muito alto

Os pesos oscilam descontroladamente. O modelo nunca converge.

Diverge!

η muito baixo

Convergência extremamente lenta. Pode ficar preso em mínimo local.

Lento demais

η adequado

Convergência suave e eficiente para o mínimo global (ou bom local).

Ideal!

OTIMIZADORES MODERNOS

Otimizador	Learning Rate Adaptativo?	Momentum?	Quando usar
SGD	Não	Opcional	Baseline simples
Adam	Sim	Sim	Padrão recomendado
RMSprop	Sim	Não	Séries temporais / RNNs



OVERFITTING & REGULARIZAÇÃO

Técnicas para evitar que o modelo memorize ao invés de generalizar

OVERFITTING vs UNDERFITTING

Underfitting

Modelo muito simples. Não captura padrões nos dados. Alta bias.

Treino: Acc: 62%
Teste: Acc: 60%

Ruim nos dois

Bom Ajuste

Modelo balanceado. Captura padrões e generaliza bem.

Treino: Acc: 88%
Teste: Acc: 85%

Equilibrado

Overfitting

Modelo memoriza treino. Não generaliza. Alta variância.

Treino: Acc: 99%
Teste: Acc: 68%

Gap enorme!

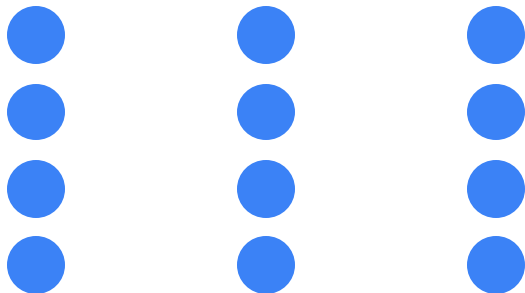


Regularização = conjunto de técnicas para combater overfitting: Dropout, Ridge (L2), Lasso (L1), Early Stopping.

DROPOUT

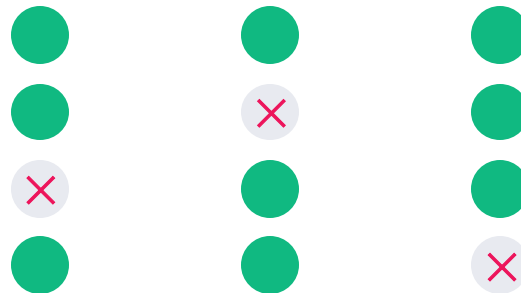
Durante o treinamento, desativa neurônios aleatoriamente em cada batch, forçando a rede a não depender de caminhos específicos.

Sem Dropout (overfitting)



Todos os neurônios ativos

Com Dropout (regularizado)



Neurônios desativados (rate=0.3)

Keras:

```
model.add(Dense(64, activation='relu'))  
model.add(Dropout(0.3)) # 30% dos neurônios desativados a cada batch
```

DROPOUT — POR QUE FUNCIONA?

1

Ensemble implícito

Cada batch treina uma sub-rede diferente. O resultado final é como um ensemble de milhares de modelos.

2

Elimina co-dependência

Neurônios não podem "confiar" em neurônios vizinhos. Cada um precisa aprender features úteis por conta própria.

3

Efeito de ruído

A desativação age como ruído (noise injection), similar a data augmentation, melhorando robustez.

4

Só no treino!

Em inferência (predict), TODOS os neurônios estão ativos, mas os pesos são escalados pelo fator $(1 - \text{rate})$.

Rates típicos: 0.2–0.5 para camadas densas. Camadas convolucionais usam 0.1–0.25. Nunca usar na camada de saída.

RIDGE — REGULARIZAÇÃO L2

Penaliza pesos grandes adicionando o quadrado dos pesos à função de perda.

$$\text{Loss total} = \text{Loss original} + \lambda \cdot \sum w_i^2$$

λ (lambda) = força da regularização. Quanto maior, mais os pesos são "encolhidos" (shrinkage) em direção a zero.

Pesos menores

Evita que um único neurônio domine a decisão

Reduz complexidade

Modelo mais simples = melhor generalização

NÃO zera pesos

Diferente do Lasso — todos os pesos continuam ativos

Keras:

```
from tensorflow.keras.regularizers import l2

model.add(Dense(64, activation='relu', kernel_regularizer=l2(0.01)))
```


LASSO — REGULARIZAÇÃO L1

Penaliza o valor absoluto dos pesos, forçando alguns a se tornarem exatamente zero — feature selection automática.

$$\text{Loss total} = \text{Loss original} + \lambda \cdot \sum |w_i|$$

A penalidade sobre $|w|$ cria gradientes constantes que "puxam" pesos pequenos diretamente para zero.

L1 (LASSO) vs L2 (RIDGE) — COMPARAÇÃO

Aspecto	Lasso (L1)	Ridge (L2)
Penalidade	$\sum w_i $	$\sum w_i^2$
Zera pesos?	Sim — feature selection!	Não — apenas encolhe
Ideal quando	Muitas features irrelevantes	Muitas features correlacionadas
Keras	<code>kernel_regularizer=l1(λ)</code>	<code>kernel_regularizer=l2(λ)</code>

Elastic Net = L1 + L2 combinados. Em Keras: `kernel_regularizer=l1_l2(l1=0.01, l2=0.01)`



EARLY STOPPING

Parando o treinamento antes que o overfitting comece

EARLY STOPPING — CONCEITO

Monitora a métrica de validação. Quando ela para de melhorar, interrompe o treinamento e restaura os melhores pesos.



patience

Nº de épocas sem melhora antes de parar. Ex: patience=10

monitor

Métrica observada. Tipicamente: val_loss ou val_accuracy

restore_best

Restaura os pesos da melhor época (não da última!)

min_delta

Melhora mínima para considerar como progresso. Ex: 0.001

No gráfico: a val_loss para de cair na época ~50 e começa a subir → OVERFITTING. Early Stopping para neste ponto.

EARLY STOPPING — IMPLEMENTAÇÃO

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

```
# Definir callbacks
early_stop = EarlyStopping(
    monitor='val_loss',      # métrica a monitorar
    patience=10,             # épocas sem melhora
    min_delta=0.001,         # melhora mínima
    restore_best_weights=True # restaura melhores pesos!
)

checkpoint = ModelCheckpoint(
    'melhor_modelo.keras',
    save_best_only=True
)

# Treinar com callbacks
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=200,              # máximo alto – Early Stopping decide!
    batch_size=32,
    callbacks=[early_stop, checkpoint])
```



Dica: defina epochs=200+ (alto) e deixe o Early Stopping decidir quando parar. Combine com ModelCheckpoint para salvar o melhor modelo.

COMBINANDO TUDO — PIPELINE COMPLETO

1

Preparar dados

Split train/val/test
Feature scaling

2

Arquitetura

Dense layers
Dropout layers

3

Regularização

L1, L2 ou
Elastic Net

4

Compilar

Adam optimizer
Binary CrossEntropy

5

Treinar

Early Stopping
ModelCheckpoint

Pipeline Completo (Keras):

```
model = Sequential([
    Dense(64, activation='relu', kernel_regularizer=l2(0.01), input_shape=(21,)),
    Dropout(0.3),
    Dense(32, activation='relu', kernel_regularizer=l2(0.01)),
    Dropout(0.2),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                    epochs=200, batch_size=32, callbacks=[early_stop])
```

PRÓXIMOS PASSOS

HANDS-ON NO JUPYTER NOTEBOOK

- ✓ Carregar dataset de churn e explorar as 21 features
- ✓ Construir pipeline completo: scaling → modelo → regularização
- ✓ Comparar modelo sem vs com regularização (Dropout + L2 + Early Stopping)
- ✓ Analisar curvas de aprendizado e métricas de negócio